

Description

Accepted

Editorial

Solutions

Submissions

All Submissions

Accepted 11511 / 11511 testcases passed

Editorial

Solution

Runtime

11 ms | Beats 6.44%

Analyze Complexity

Memory

12.26 MB | Beats 5.07%



Code

C++

```
1 class Solution {
2 public:
3     bool isPalindrome(int x) {
4         if (x < 0){return false;} // if it is false it is not a Palindrome
5
6         //challage accepted (not converting the number to a sting)
7         // getting the didgts as an array by using the % operator by 10
8         // as this is int division the last digit/10 = 0
9
10        //comparing the first and last element and moving in towards the middle
11        for(int i = 0 , j = digits.size() - 1; i < digits.size() ; i++ , j -- ){
12            if (digits[i] != digits[j]) {return false;}
13        }
14
15        return true;
16    }
17 };
```

More challenges

234. Palindrome Linked List

2396. Strictly Palindromic Number

3260. Find the Largest Palindrome Divisible by K

Write your notes here

Code

C++ Auto

```
1 class Solution {
2 public:
3     bool isPalindrome(int x) {
4         if (x < 0){return false;} // if it is false it is not a Palindrome
5
6         //challage accepted (not converting the number to a sting)
7         // getting the didgts as an array by using the % operator by 10
8         // as this is int division the last digit/10 = 0
9         vector<int> digits = {};
10        while(x != 0){
11            digits.push_back(x%10);
12            x = x / 10;
13        }
14
15        //comparing the first and last element and moving in towards the middle
16        for(int i = 0 , j = digits.size() - 1; i < digits.size() ; i++ , j -- ){
17            if (digits[i] != digits[j]) {return false;}
18        }
19
20        return true;
21    }
22 };
```

Saved

Ln 22, Col 3

Testcase

Test Result

DescriptionAccepted x Editorial SolutionsSubmissions

All Submissions

Accepted114 / 114 testcases passed

yousab21 submitted at Feb 18, 2026 12:52

EditorialSolution

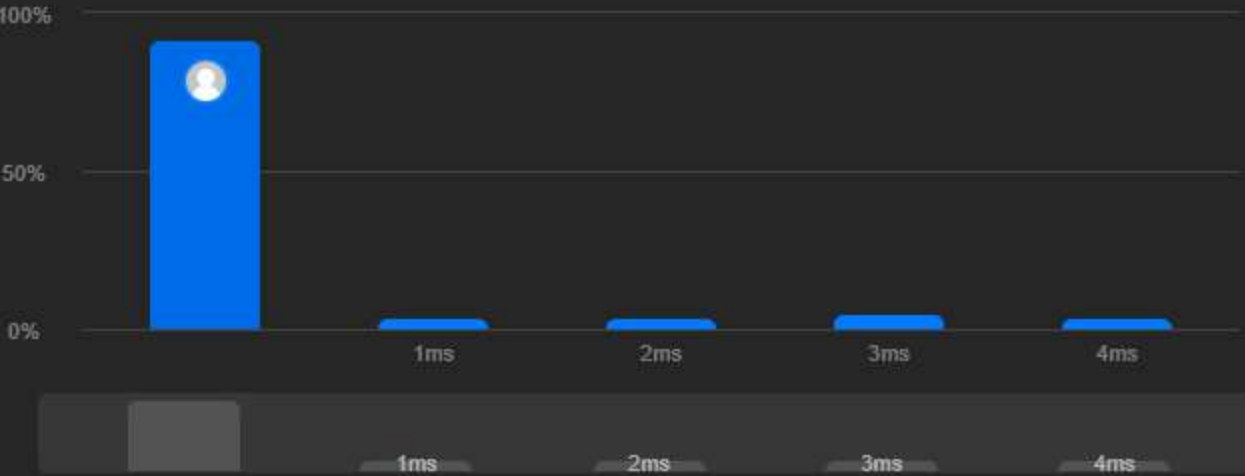
Runtime

0 ms | Beats 100.00%

Analyze Complexity

Memory

11.66 MB | Beats 18.25%



Runtime	Memory
0 ms	11.66 MB
1 ms	
2 ms	
3 ms	
4 ms	

CodeC++

```
1 class Solution {
2 public:
3     vector<int> plusOne(vector<int>& digits) {
4         for(int i = digits.size()- 1 ; i < digits.size() ; i--){
5             // if we can 1 to the digit we are on just add it and quit
6             if(digits[i] < 9){
7                 digits[i]++ ;
8                 break;
9             }
10            // if the digit is 9 (we cant add 1) make the digit 0
11            // and go to the next digit and try to add one to that
12            else if (digits[i] = 9 && i !=0 ){
13                digits[i] = 0;
14                continue;
15            }
16            // if we reached this point that means the whole number is made of 9s
17            // so we make all didigits 0 and creat a new digit with 1
18            else if (digits[i] = 9 && i ==0 ){
19                digits[i] = 0;
20                digits.insert(digits.begin(), 1);
21                break;
22            }
23        }
24        return digits;
25    }
26 };
```

View more

More challenges

369. Plus One Linked List

989. Add to Array-Form of Integer

2571. Minimum Operations to Reduce an Integer to 0

Write your notes here

Code

C++Auto

```
1 class Solution {
2 public:
3     vector<int> plusOne(vector<int>& digits) {
4         for(int i = digits.size()- 1 ; i < digits.size() ; i--){
5             // if we can 1 to the digit we are on just add it and quit
6             if(digits[i] < 9){
7                 digits[i]++ ;
8                 break;
9             }
10            // if the digit is 9 (we cant add 1) make the digit 0
11            // and go to the next digit and try to add one to that
12            else if (digits[i] = 9 && i !=0 ){
13                digits[i] = 0;
14                continue;
15            }
16            // if we reached this point that means the whole number is made of 9s
17            // so we make all didigits 0 and creat a new digit with 1
18            else if (digits[i] = 9 && i ==0 ){
19                digits[i] = 0;
20                digits.insert(digits.begin(), 1);
21                break;
22            }
23        }
24        return digits;
25    }
26 };
```

Saved

Ln 30, Col 16

TestcaseTest Result

DescriptionAccepted x Editorial SolutionsSubmissions

All Submissions

Accepted126 / 126 testcases passed

yousab21 submitted at Feb 18, 2026 13:36

EditorialSolution

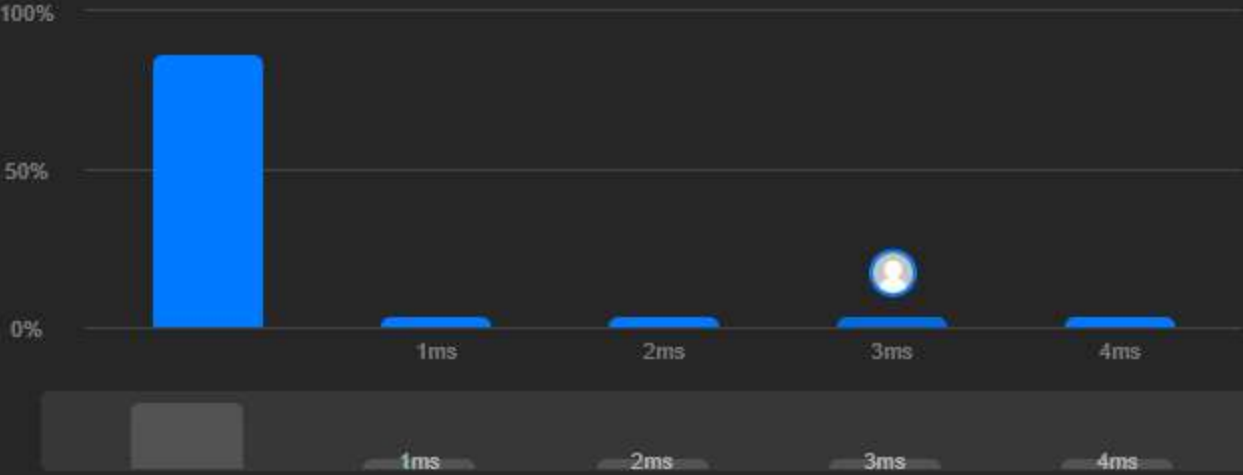
Runtime

3 ms | Beats 10.59%

Analyze Complexity

Memory

18.20 MB | Beats 5.04%



CodeC++

```
1 class Solution {
2 public:
3     string longestCommonPrefix(vector<string>& strs) {
4
5         //getting the smallest string because :
6         //1- the longest common prefix must be a part of it
7         //2- iam afraid of going out of bounds if i start with a longer stri
8         string shortest = strs[0];
9
10        // for doing the comparing:
11        // 1- we go over every char in the shortest word
12        // 2- we compare that to the equevilant char in every other word
13        // 3- if it is there we add it to our prefix
14        // 4- if it is not there we raise a flag so doesnt get added and we leave the loop
15
16        string prefix = "";
17        bool CharIsInAll = true;
18        for(int i = 0; i<shortest.size(); i++){
19
20            for(string word : strs){
21                if(word[i] != shortest[i]){
22                    CharIsInAll = false;
23                }
24            }
25
26            if(CharIsInAll){
27                prefix.push_back(shortest[i]);
28            }
29            else{break;}
30        }
31
32        return prefix;
33    };
34 }
```

View more

More challenges

2996. Smallest Missing Integer Greater Than Sequential Prefix Sum

3093. Longest Common Suffix Queries

3460. Longest Common Prefix After at Most One Removal

Write your notes here

Code

C++ v Auto

```
1 class Solution {
2 public:
3     string longestCommonPrefix(vector<string>& strs) {
4
5         //getting the smallest string because :
6         //1- the longest common prefix must be a part of it
7         //2- iam afraid of going out of bounds if i start with a longer string
8         string shortest = strs[0];
9         for(string i : strs){
10             if (i.size() < shortest.size()){
11                 shortest = i;
12             }
13         }
14
15         // for doing the comparing:
16         // 1- we go over every char in the shortest word
17         // 2- we compare that to the equevilant char in every other word
18         // 3- if it is there we add it to our prefix
19         // 4- if it is not there we raise a flag so doesnt get added and we leave the loop
20
21         string prefix = "";
22         bool CharIsInAll = true;
23         for(int i = 0; i<shortest.size(); i++){
24
25             for(string word : strs){
26                 if(word[i] != shortest[i]){
27                     CharIsInAll = false;
28                 }
29             }
30
31             if(CharIsInAll){
32                 prefix.push_back(shortest[i]);
33             }
34             else{break;}
35         }
36
37         return prefix;
38     };
39 }
```

Saved

Ln 37, Col 7

Testcase> Test Result